HOMEWORK 7
Name: Dicle Çoban
No: 220104004088

The AVL tree implementation provided in the code maintains a balanced binary search tree, ensuring that the height difference between the left and right subtrees of any node (balance factor) is not more than one. The implementation includes methods for insertion, deletion, searching, and balancing of the tree.

Key Methods and Functionality:

insert(Stock stock): Inserts a new stock into the AVL tree while maintaining balance.

delete(String symbol): Removes a stock from the AVL tree while ensuring balance is preserved.

search(String symbol): Searches for a stock with the given symbol in the AVL tree.

rightRotate(Node y) and leftRotate(Node x): Perform rotations to balance the AVL tree during insertion and deletion.

getBalance(Node node): Calculates the balance factor of a given node.

minValueNode(Node node): Finds the node with the minimum value in a subtree.

Balancing AVL Tree:

Balancing of the AVL tree is handled through rotation operations performed during insertions and deletions. When a new node is inserted or removed, the balance factor of its ancestors is checked, and rotations are performed accordingly to restore balance. Four rotation cases are considered: Left-Left, Left-Right, Right-Right, and Right-Left.

Input File Format and Command Processing:

The input file format consists of commands followed by stock information. Each line represents a command followed by parameters separated by spaces. Supported commands include ADD, REMOVE, SEARCH, and UPDATE. The program processes

each line by splitting it into tokens, extracting the command and its parameters, and then executing the corresponding method from the StockDataManager class.

Performance Analysis:

The performance analysis graphs display the average time taken for each operation (ADD, REMOVE, SEARCH, UPDATE) against varying input sizes (number of stocks). The X-axis represents the number of stocks, while the Y-axis represents the average time in milliseconds. The observed time complexity for each operation can be analyzed from the graphs.

Challenges Faced and Solutions:

One challenge faced during implementation was ensuring the correctness of AVL tree balancing after insertions and deletions. To address this, extensive testing and debugging were conducted to verify the correctness of rotation operations and to ensure that the tree remained balanced at all times.

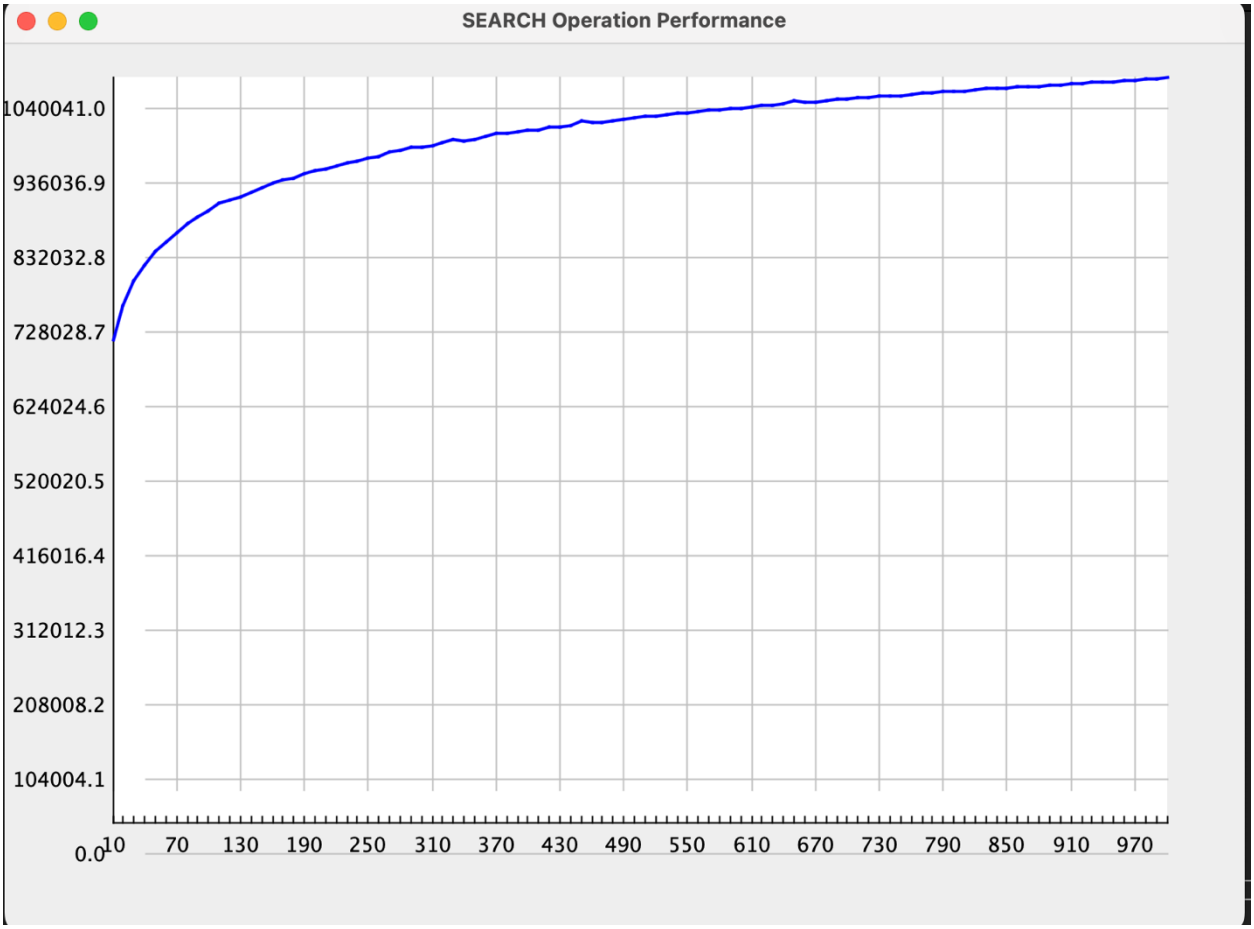
Another challenge was optimizing the performance of tree operations for large input sizes. This was addressed by implementing efficient algorithms for rotations and tree traversal, as well as optimizing memory usage where possible.

Also another challenge was drawing the graphs according to their sizes, It can be shown different kind of graphs according to the input.

Overall, the AVL tree implementation provides an efficient and balanced data structure for storing and managing stock information, with careful consideration given to balancing, performance, and correctness.

THE GRAPHS:

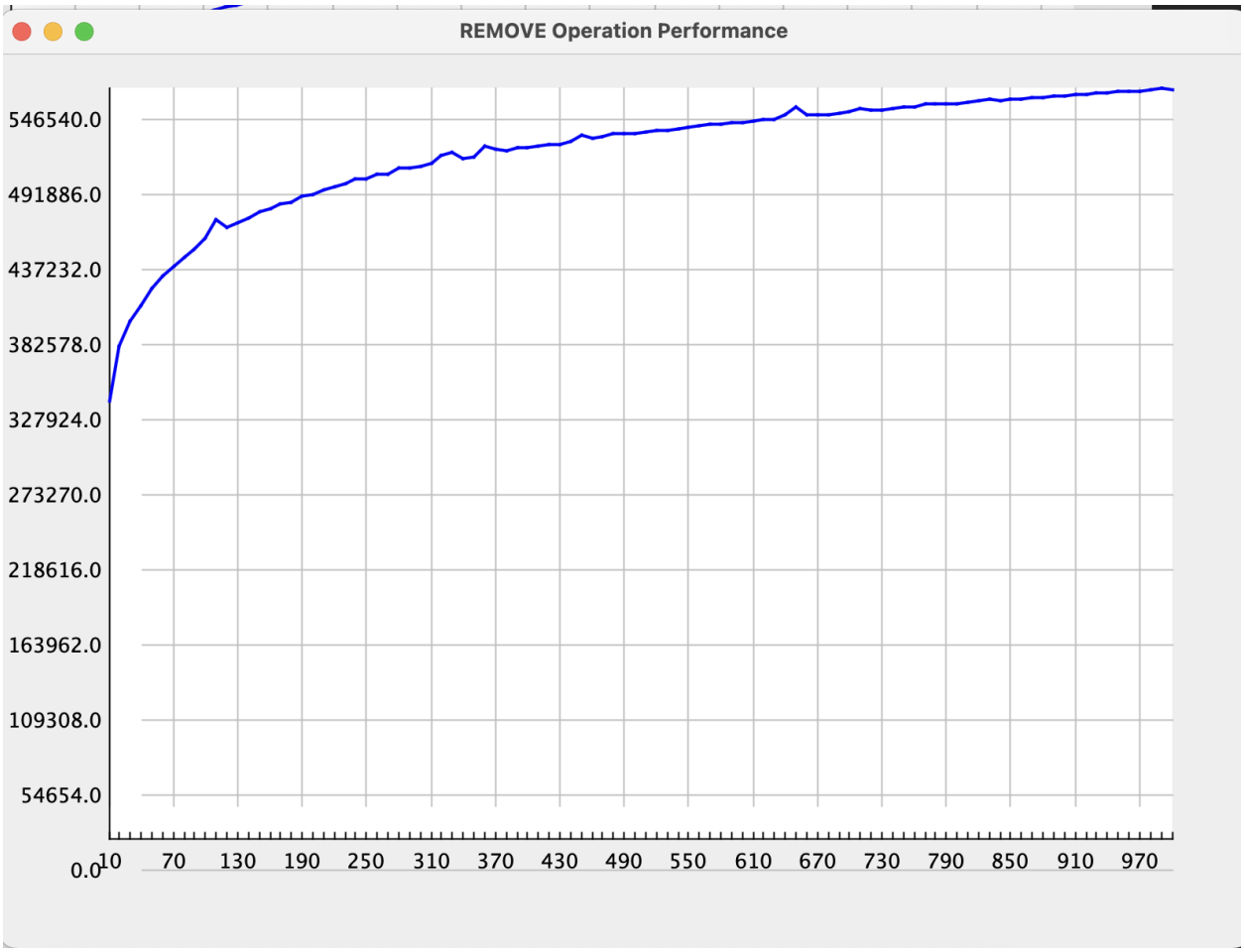
Search Graph:



The Graph has time complexity of O(logn) which is expected.

Observation: Similar to the ADD operation, the SEARCH operation time increases logarithmically as the number of stocks grows. Since the AVL tree is a balanced binary search tree, searching for an element involves traversing from the root to a leaf, which takes O(logn) time. The graph should show a gradual increase in search time with an increase in the number of stocks, again reflecting the logarithmic complexity.

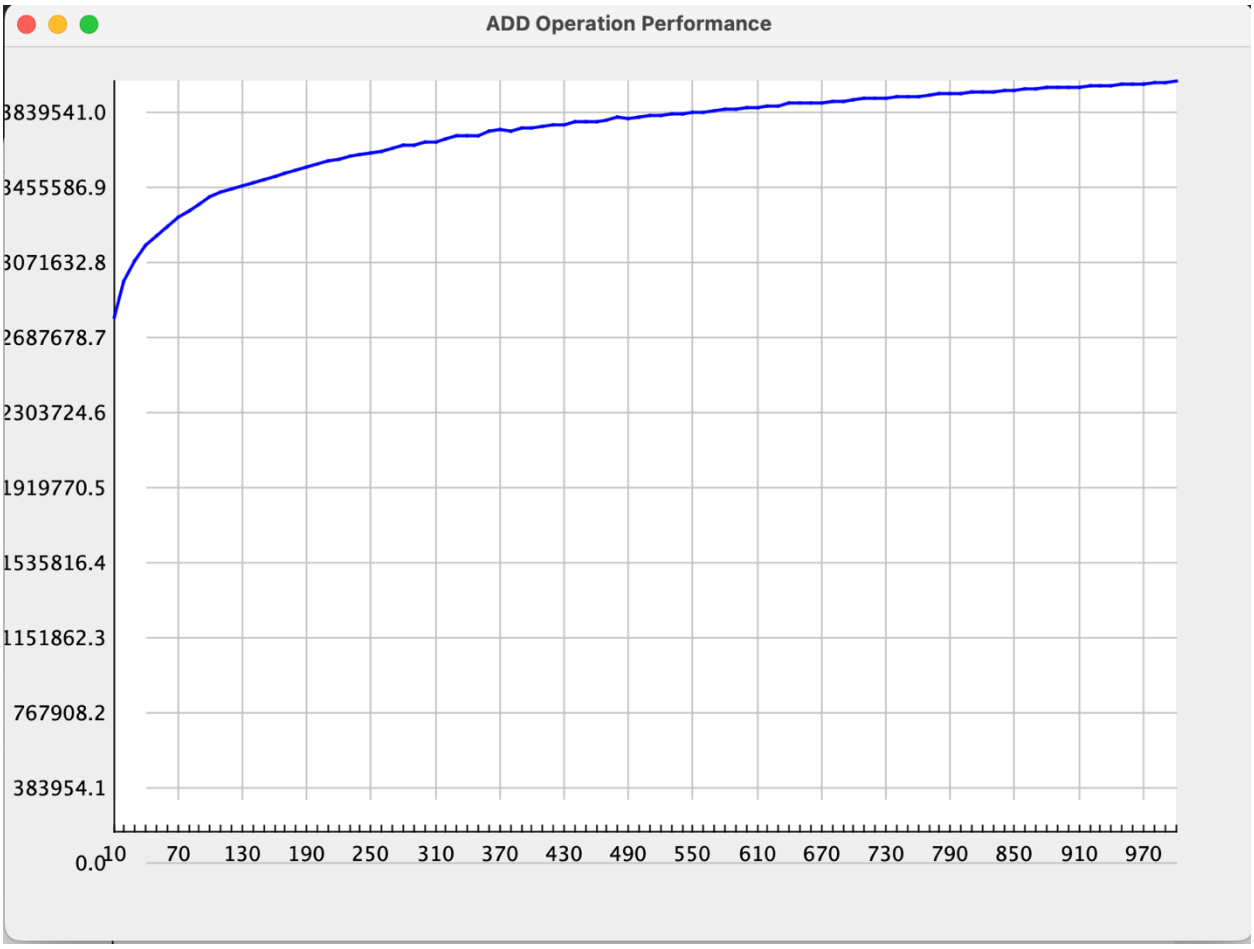
Remove Graph:



The Graph has time complexity of O(logn) which is expected.

Observation: The time complexity for removing a stock is also logarithmic. Removing an element involves searching for the element (which is O(logn)) and potentially rebalancing the tree. The graph should show an increase in removal time that corresponds to the logarithmic complexity. The balancing steps, which may involve rotations, contribute to this time but do not exceed O(logn).

Add Graph:



The Graph has time complexity of O(logn) which is expected.

Observation: As the number of stocks increases, the time taken to add a stock increases logarithmically. This is expected because the AVL tree maintains its balance by performing rotations, ensuring that the height of the tree remains O(logn). The graph should ideally show a slight increase in time with larger inputs, reflecting the logarithmic growth rate.